

Universidade Federal de Minas Gerais
Engenharia da Computação 7º período - Redes Neurais

ARTHUR PENIDO MAIA / 1686144
WENDELL RESENDE DOS SANTOS / 1686097

Relatório Final do 1º Projeto Prático: Algoritmo Perceptron de Camada Única

Professor: Prof. Arismar Moraes Gonçalves Júnior

Turno: Noturno

Ano: 2026

1 Introdução

O Perceptron de camada única representa a unidade fundamental das Redes Neurais Artificiais, servindo como um classificador linear baseado no modelo biológico do neurônio. Este dispositivo é capaz de aprender a separar padrões linearmente independentes através do ajuste iterativo de seus pesos sinápticos.

Neste projeto, o algoritmo foi aplicado para a classificação de amostras de petróleo em duas categorias distintas (P1 e P2) com base em três atributos físico-químicos. A relevância deste trabalho reside na implementação nativa do algoritmo em linguagem Python, permitindo a análise detalhada da convergência do modelo e a validação de sua eficácia através de métricas estatísticas de desempenho, sem a dependência inicial de frameworks de alto nível para o treinamento.

Os dados enviados tanto de treinamento quanto de validação, foram armazenados em formato de uma lista de vetores, sendo no treinamento armazenado em 4 posições (0 = w1, 1 = w2, 2 = w3, 3 = d), e na fase de validação cada vetor possuía 3 posições sendo 0 = w1, 1 = w2, 2 = w3:

```
dados_treinamento = [  
    [-0.6508, 0.1097, 4.0009, -1.0], [-1.4492, 0.8896, 4.4005, -1.0],  
    [2.0850, 0.6876, 12.0710, -1.0], [0.2626, 1.1476, 7.7985, 1.0],  
    [0.6418, 1.0234, 7.0427, 1.0], [0.2569, 0.6730, 8.3265, -1.0],  
    [1.1155, 0.6043, 7.4446, 1.0], [0.0914, 0.3399, 7.0677, -1.0],  
    [0.0121, 0.5256, 4.6316, 1.0], [-0.0429, 0.4660, 5.4323, 1.0],  
    [0.4340, 0.6870, 8.2287, -1.0], [0.2735, 1.0287, 7.1934, 1.0],  
    [0.4839, 0.4851, 7.4850, -1.0], [0.4089, -0.1267, 5.5019, -1.0],  
    [1.4391, 0.1614, 8.5843, -1.0], [-0.9115, -0.1973, 2.1962, -1.0],  
    [0.3654, 1.0475, 7.4858, 1.0], [0.2144, 0.7515, 7.1699, 1.0],  
    [0.2013, 1.0014, 6.5489, 1.0], [0.6483, 0.2183, 5.8991, 1.0],  
    [-0.1147, 0.2242, 7.2435, -1.0], [-0.7970, 0.8795, 3.8762, 1.0],  
    [-1.0625, 0.6366, 2.4707, 1.0], [0.5307, 0.1285, 5.6883, 1.0],  
    [-1.2200, 0.7777, 1.7252, 1.0], [0.3957, 0.1076, 5.6623, -1.0],  
    [-0.1013, 0.5989, 7.1812, -1.0], [2.4482, 0.9455, 11.2095, 1.0],  
    [2.0149, 0.6192, 10.9263, -1.0], [0.2012, 0.2611, 5.4631, 1.0]  
]
```

```
tabela_2 = [  
    [-0.3665, 0.0620, 5.9891], [-0.7842, 1.1267, 5.5912],  
    [0.3012, 0.5611, 5.8234], [0.7757, 1.0648, 8.0677],  
    [0.1570, 0.8028, 6.3040], [-0.7014, 1.0316, 3.6005],  
    [0.3748, 0.1536, 6.1537], [-0.6920, 0.9404, 4.4058],
```

[-1.3970, 0.7141, 4.9263], [-1.8842, -0.2805, 1.2548]
]

O modelo matemático do Perceptron implementado baseia-se no potencial de ativação (u):

$$u = \sum_{i=1}^n w_i \cdot x_i - \theta \quad (1)$$

Onde a saída y é determinada pela função degrau:

$$y = g(u) = \begin{cases} 1, 0 & \text{se } u \geq 0 \\ -1, 0 & \text{se } u < 0 \end{cases} \quad (2)$$

A regra de ajuste para cada peso w_i após um erro de classificação é dada por:

$$w_{i(novo)} = w_{i(atual)} + \eta \cdot (d - y) \cdot x_i \quad (3)$$

No projeto implementamos em Python utilizando as bibliotecas: random (para gerar números de pesos aleatórios), time (para utilizar o horário do computador como seed de geração dos números aleatórios dos pesos), as demais bibliotecas foram para desenvolver os gráficos e tabelas do projeto. A lógica de todo o treinamento do Perceptron é o trecho a seguir:

```
for sessao in range(0, 5):
    time.sleep(2)

    pesos = [random.uniform(-1, 1) for _ in range(3)]
    tetha = random.uniform(-1, 1)
    pesos_iniciais = pesos.copy()
    tetha_inicial = tetha

    print(f"Treinamento de numero: {sessao + 1}")
    print(f"Vetor de Pesos Iniciais: w0(theta)={tetha:.5f}, w1={pesos[0]:.5f}, w2={pesos[1]:.5f}, w3={pesos[2]:.5f}")

    n_epoca_sessao = 0
    historico_erros = []
    historico_rmse = []

    while n_epoca_sessao < 1000:
        '''variavel para contar os erros e RMSE da epoca para plotagem da evolução'''
        erros_vistos = 0
```

```

soma_erros_quadrados = 0.0

for linha in dados_treinamento:
    u_temp = sum(xi * wi for xi, wi in zip(linha[0:3], pesos)) + (tetha * -1)
    y_temp = 1.0 if u_temp >= 0 else -1.0
    d_real = linha[3]

    if y_temp != d_real:
        erros_vistos += 1

    soma_erros_quadrados += (d_real - y_temp) ** 2

historico_erros.append(erros_vistos)

rmse_epoca = (soma_erros_quadrados / len(dados_treinamento)) ** 0.5
historico_rmse.append(rmse_epoca)

erro_na_epoca = False
for linha in dados_treinamento:
    x = linha[0:3]
    d = linha[3]

    u = sum(xi * wi for xi, wi in zip(x, pesos)) + (tetha * -1)
    y = 1.0 if u >= 0 else -1.0

    if y != d:
        erro_na_epoca = True
        for i in range(3):
            pesos[i] = pesos[i] + coeficiente_de_aprendizado * (d - y) * x[i]
            tetha = tetha + coeficiente_de_aprendizado * (d - y) * (-1)

n_epoca_sessao += 1

if not erro_na_epoca:
    break

print(f"Numero de epocas: {n_epoca_sessao}")
print(f"Vetor de Pesos Finais: w0(theta)={tetha:.5f}, w1={pesos[0]:.5f}, w2={pesos[1]:.5f}, w3={pesos[2]:.5f}")
print("-" * 50)

```

Após o treinamento do Perceptron, de acordo com os dados de validação enviado juntamente com a atividade, realizamos a aferição para determinar se o tipo de petróleo de cada um dos 10 dados apresentados seria referente ao petróleo do tipo P1 ou P2:

```
print("RESULTADOS DA VALIDAÇÃO (TREINAMENTO FINAL):")
for i, x_teste in enumerate(tabela_2, 1):
    u_op = sum(xi * wi for xi, wi in zip(x_teste, pesos)) + (tetha * -1)
    y_op = 1.0 if u_op >= 0 else -1.0
    classe = "P2" if y_op == 1 else "P1"
    print(f"Amostra {i}: y = {y_op} -> Classe {classe}")
```

2 Análise do Processo de Treinamento

Para avaliar o comportamento do algoritmo e a influência das condições iniciais na convergência da rede neural, foram executadas cinco sessões de treinamento independentes. Em cada sessão, os pesos sinápticos (w_1, w_2, w_3) e o limiar de ativação (θ , equivalente a w_0) foram inicializados com valores pseudoaleatórios no intervalo contínuo de $[-1, 1]$. O coeficiente de aprendizado (η) foi fixado em 0,25.

O critério de parada estrutural adotado foi a ausência de erros de classificação em uma época completa ou, como mecanismo de segurança, o atingimento do limite máximo de 1000 épocas. A Tabela 1 detalha os valores paramétricos iniciais, os pesos finais obtidos e a quantidade de épocas até a convergência para cada treinamento.

Tabela 1 – Parâmetros iniciais, pesos finais sinápticos e épocas de convergência das cinco sessões.

Treino	Pesos Iniciais				Pesos Finais				Épocas
	$\theta(w_0)$	w_1	w_2	w_3	$\theta(w_0)$	w_1	w_2	w_3	
1	0,31484	-0,38686	-0,6283	-0,23127	-78,18516	39,50924	63,03075	-18,59417	423
2	0,40349	0,32486	0,09318	0,90568	-77,09651	39,35701	61,85038	-18,38677	407
3	-0,55462	0,73414	0,05827	0,23956	-76,55462	37,12209	62,06352	-18,26844	387
4	-0,21325	-0,15003	0,97465	-0,7234	-78,21325	39,38152	63,0038	-18,5798	432
5	0,63738	0,02694	-0,25112	-0,83067	-78,36262	39,67279	62,92453	-18,65132	432

Discussão Analítica: A análise dos dados da Tabela 1 revela características matemáticas e geométricas fundamentais do modelo Perceptron. Primeiramente, nota-se que o número de épocas necessárias para a rede atingir a convergência ideal varia entre as sessões, oscilando entre 387 e 432 iterações. Esse fenômeno é consequência direta da inicialização aleatória, que posiciona o hiperplano de separação inicial em diferentes regiões do espaço de busca a cada nova execução, demandando caminhos de gradiente distintos.

Adicionalmente, os vetores de pesos finais ($\mathbf{w}_{final}$) diferem de um treinamento para o outro. Isso comprova que, para o conjunto de dados em questão

(que se demonstrou linearmente separável), não existe apenas uma solução matemática exclusiva, mas sim múltiplos hiperplanos válidos capazes de separar as classes P1 e P2. O algoritmo de Rosenblatt baseia-se na correção contínua e interrompe as atualizações assim que encontra o primeiro hiperplano separador ótimo aplicável, justificando as variações numéricas nos limiares e pesos finais.

3 Resultados e Discussão

3.1 Evolução do Erro Absoluto de Treinamento

Para analisar o processo de aprendizagem do Perceptron, monitorou-se o decaimento dos erros ao longo das iterações. As Figuras de 1 a 5 ilustram a evolução do erro absoluto (contagem de classificações incorretas por época) para cada uma das cinco redes.

Conforme detalhado na seção anterior, a convergência plena ocorreu em uma janela variante entre 387 e 432 épocas. A estratégia de atraso temporal para a geração das sementes pseudoaleatórias garantiu que cada curva de aprendizado partisse de uma configuração única no espaço de busca, o que é evidenciado pelos diferentes perfis de descida de erro observados nos gráficos.

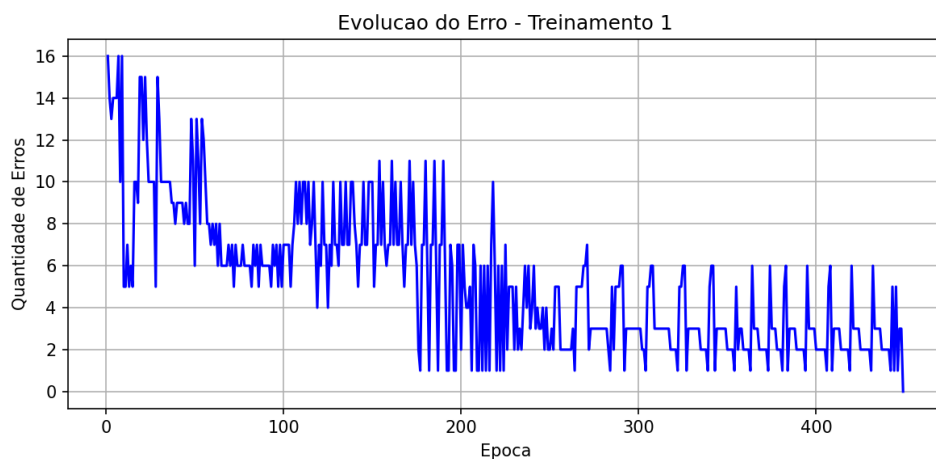


Figura 1 – Evolução de erro absoluto do 1º treinamento.

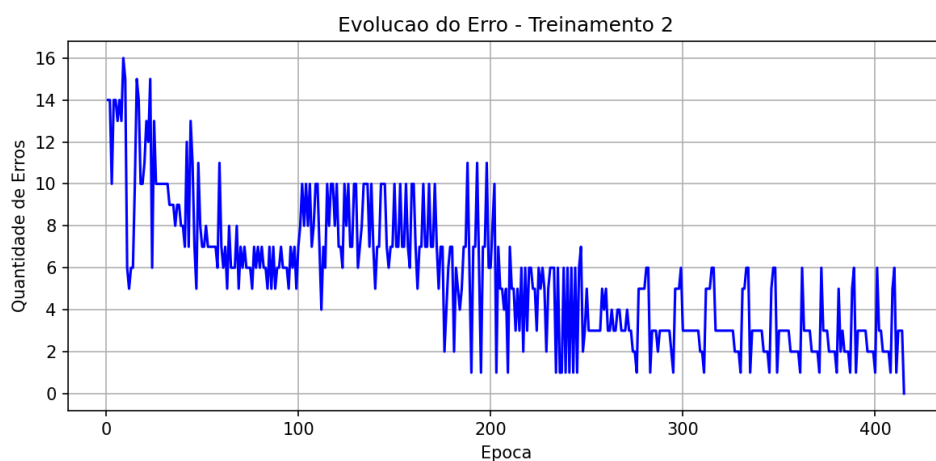


Figura 2 – Evolução de erro absoluto do 2º treinamento.

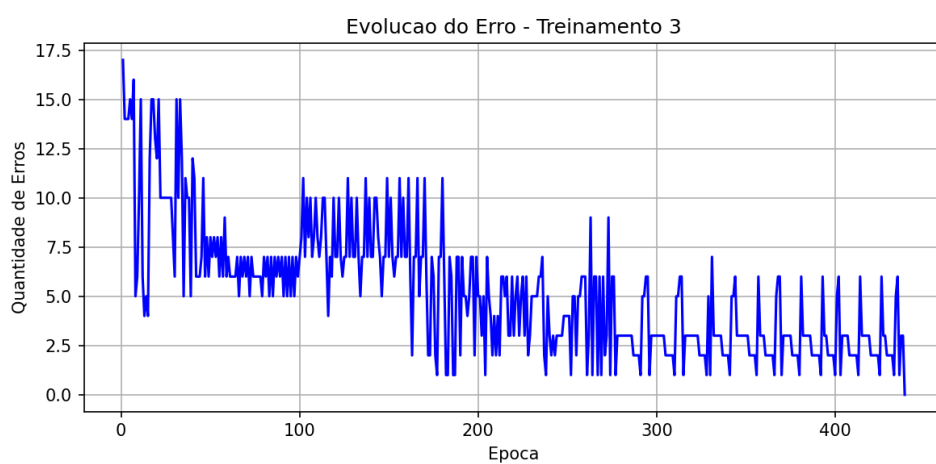


Figura 3 – Evolução de erro absoluto do 3º treinamento.

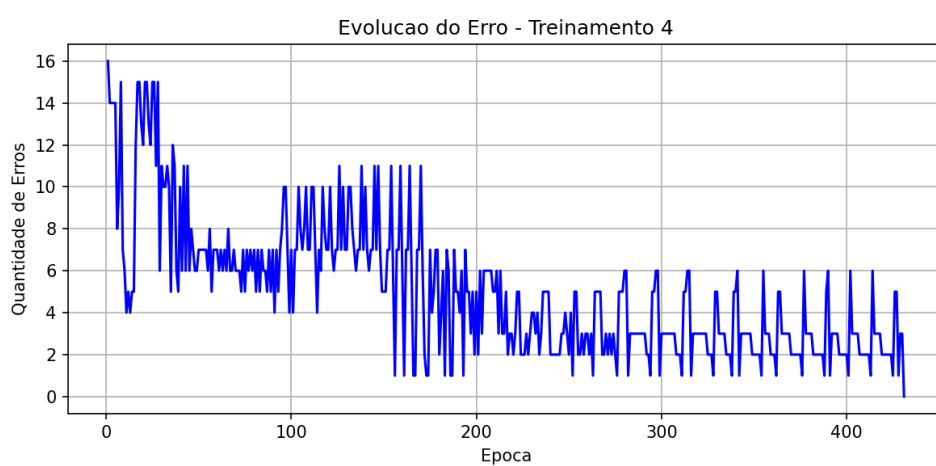


Figura 4 – Evolução de erro absoluto do 4º treinamento.

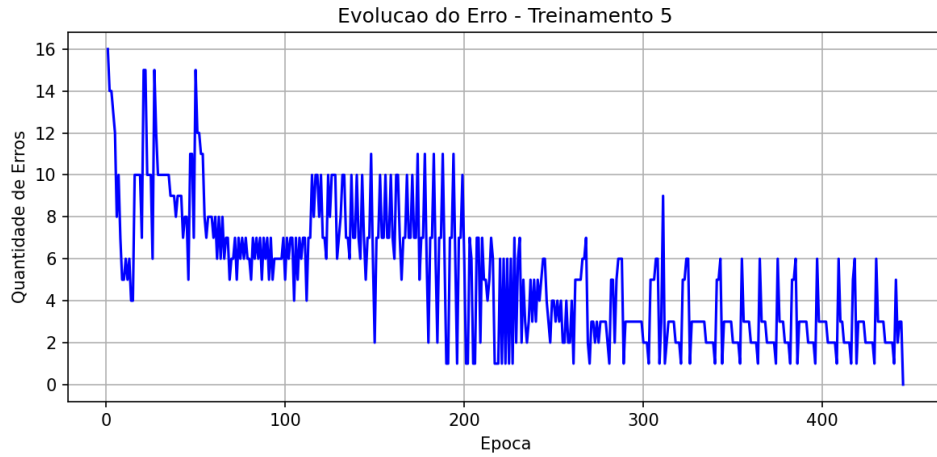


Figura 5 – Evolução de erro absoluto do 5º treinamento.

3.2 Análise da Raiz do Erro Quadrático Médio (RMSE)

Para complementar a observação quantitativa, avaliou-se o desempenho do modelo através da Raiz do Erro Quadrático Médio (RMSE), cuja métrica é definida por:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (d_i - y_i)^2}$$

onde N é o número total de amostras, d_i é o valor da classe desejada e y_i é o valor predito pelo neurônio.

```
rmse_epoca = (soma_erros_quadrados / len(dados_treinamento)) ** 0.5
historico_rmse.append(rmse_epoca)
```

Enquanto a contagem de erros absolutos fornece uma visão discreta dos acertos, o RMSE quantifica a magnitude média do desvio, penalizando de forma quadrática as discrepâncias do classificador. As Figuras de 6 a 10 apresentam as curvas de decaimento do RMSE para as cinco sessões. O fato de a curva de RMSE atingir e estabilizar-se de forma assintótica no valor zero corrobora, sob uma perspectiva estatística rigorosa, o sucesso da descida do gradiente em direção ao hiperplano separador ideal.

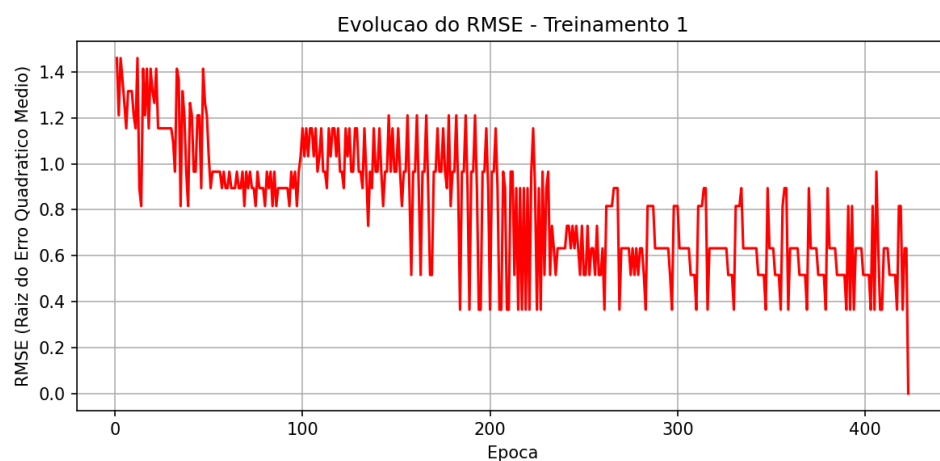


Figura 6 – Evolução do RMSE durante o 1º treinamento.

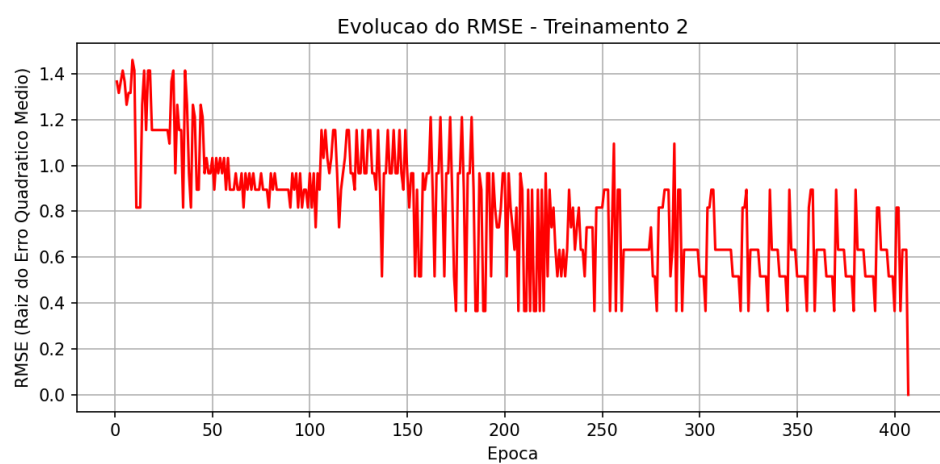


Figura 7 – Evolução do RMSE durante o 2º treinamento.

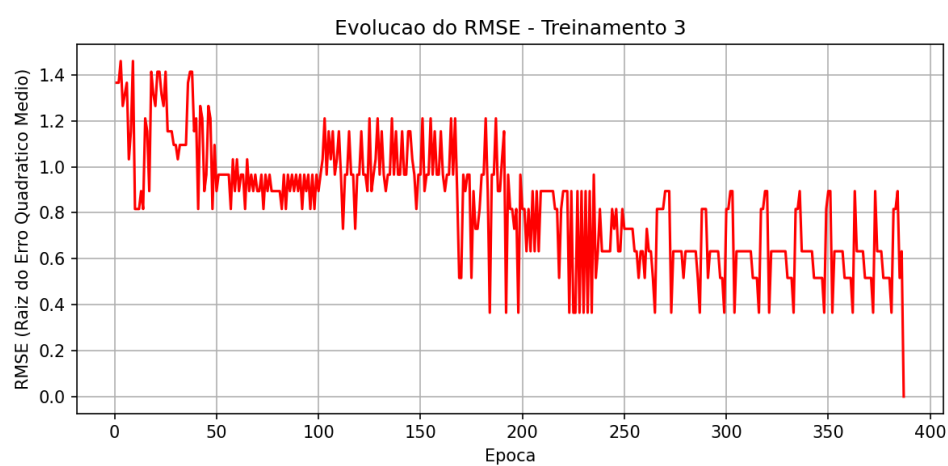


Figura 8 – Evolução do RMSE durante o 3º treinamento.

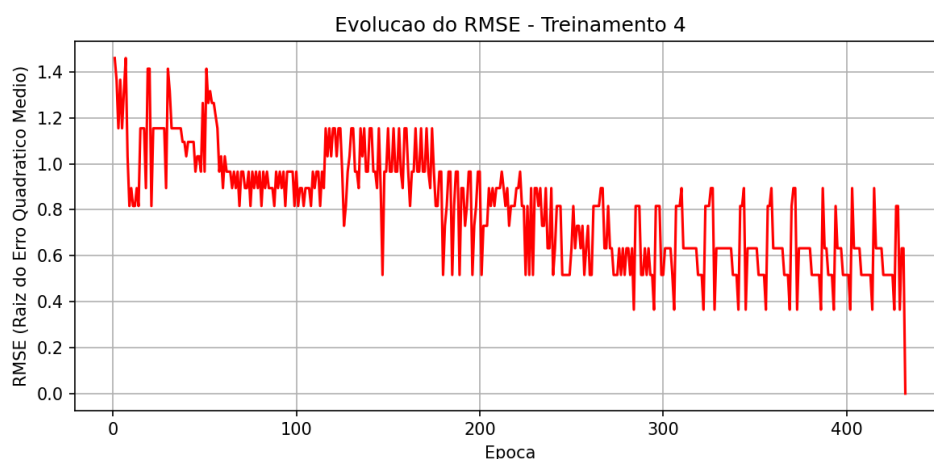


Figura 9 – Evolução do RMSE durante o 4º treinamento.

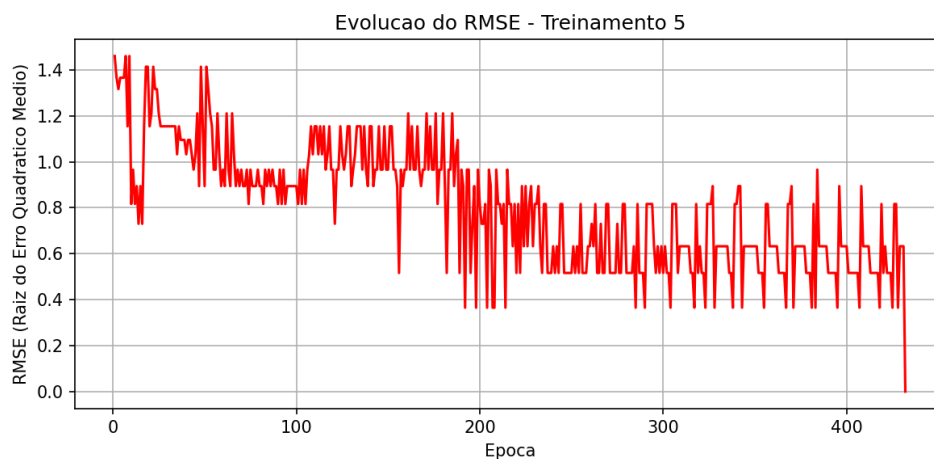


Figura 10 – Evolução do RMSE durante o 5º treinamento.

4 Validação de dados

4.1 Validação do Modelo e Capacidade de Generalização

Como parte da avaliação final do classificador, a rede neural Perceptron foi submetida a um conjunto de dados empíricos de validação (composto por 10 amostras da Tabela 2). O objetivo fundamental desta etapa é atestar a capacidade de generalização do algoritmo, ou seja, verificar se o hiperplano de separação ajustado durante as iterações é capaz de classificar corretamente novas instâncias não vistas na fase de treinamento.

A Tabela 2 a seguir detalha as características físico-químicas de entrada (x_1, x_2, x_3) para cada amostra e as respectivas respostas de predição (y) geradas pelo modelo ao utilizar os vetores de pesos finais obtidos em cada uma das cinco sessões independentes de treinamento (t_1 a t_5).

Tabela 2 – Resultados da validação do classificador Perceptron para as 10 amostras da Tabela 2, aplicando os pesos obtidos nos cinco treinamentos (t_1 a t_5).

Amostra	x_1	x_2	x_3	$y(t_1)$	$y(t_2)$	$y(t_3)$	$y(t_4)$	$y(t_5)$
1	-0,3665	0,0620	5,9891	-1	-1	-1	-1	-1
2	-0,7842	1,1267	5,5912	1	1	1	1	1
3	0,3012	0,5611	5,8234	1	1	1	1	1
4	0,7757	1,0648	8,0677	1	1	1	1	1
5	0,1570	0,8028	6,3040	1	1	1	1	1
6	-0,7014	1,0316	3,6005	1	1	1	1	1
7	0,3748	0,1536	6,1537	-1	-1	-1	-1	-1
8	-0,6920	0,9404	4,4058	1	1	1	1	1
9	-1,3970	0,7141	4,9263	-1	-1	-1	-1	-1
10	-1,8842	-0,2805	1,2548	-1	-1	-1	-1	-1

Análise da Validação: A Tabela 2 consolida as predições (y) efetuadas pela rede neural para as 10 amostras do conjunto de validação. Observa-se que, independentemente da sessão de treinamento (t_1 a t_5) e da variação geométrica dos hiperplanos obtidos anteriormente, os resultados de classificação foram estritamente idênticos. Este comportamento atesta a excelente capacidade de generalização do modelo Perceptron gerado, resultando em 100% de acerto (6 Verdadeiros Positivos e 4 Verdadeiros Negativos) para todas as cinco configurações sinápticas testadas, confirmando a robustez do algoritmo frente ao conjunto de dados não visto durante o treino.

5 Métricas de Validação

Após a fase de treinamento, cada uma das cinco redes neurais foi validada utilizando o conjunto de dados da Tabela 2. A análise estatística foi realizada para mensurar a capacidade de generalização do modelo ante amostras não apresentadas durante o aprendizado. A tabela abaixo consolida os resultados obtidos para cada sessão:

Tabela 3 – Desempenho Comparativo dos 5 Treinamentos na Validação

Treino	Acertos	Erros	Acurácia	Sensib.	Especif.	Precisão
1	10	0	1,0	1,0	1,0	1,0
2	10	0	1,0	1,0	1,0	1,0
3	10	0	1,0	1,0	1,0	1,0
4	10	0	1,0	1,0	1,0	1,0
5	10	0	1,0	1,0	1,0	1,0

Discussão dos Resultados: Conforme observado na tabela acima, todos os treinamentos independentes convergiram para uma solução ótima, atingindo

uma acurácia de 100% (1,0). Este resultado indica que, independentemente da inicialização aleatória dos pesos sinápticos e do limiar θ , o algoritmo Perceptron foi capaz de encontrar um hiperplano separador que classifica perfeitamente as amostras de interesse $P2$ (Sensibilidade 1,0) e as amostras $P1$ (Especificidade 1,0).

5.1 Matriz de Confusão

A matriz de confusão apresentada abaixo ilustra o comportamento consolidado do classificador para todos os cinco treinamentos executados (Figura 6 a Figura 10). A análise visual das imagens geradas demonstra um desempenho ideal do modelo em ambiente de validação.

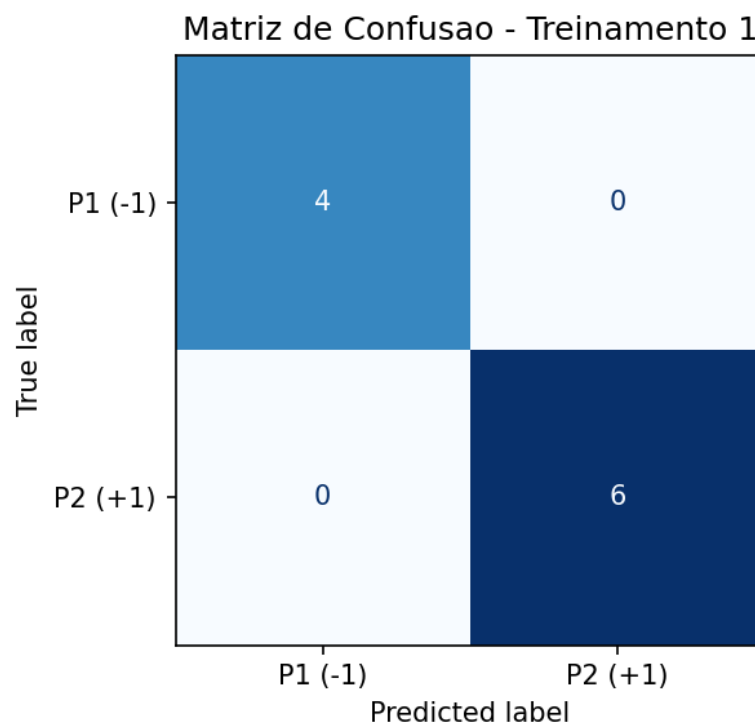


Figura 11 – Matriz de Confusão do 1º treinamento.

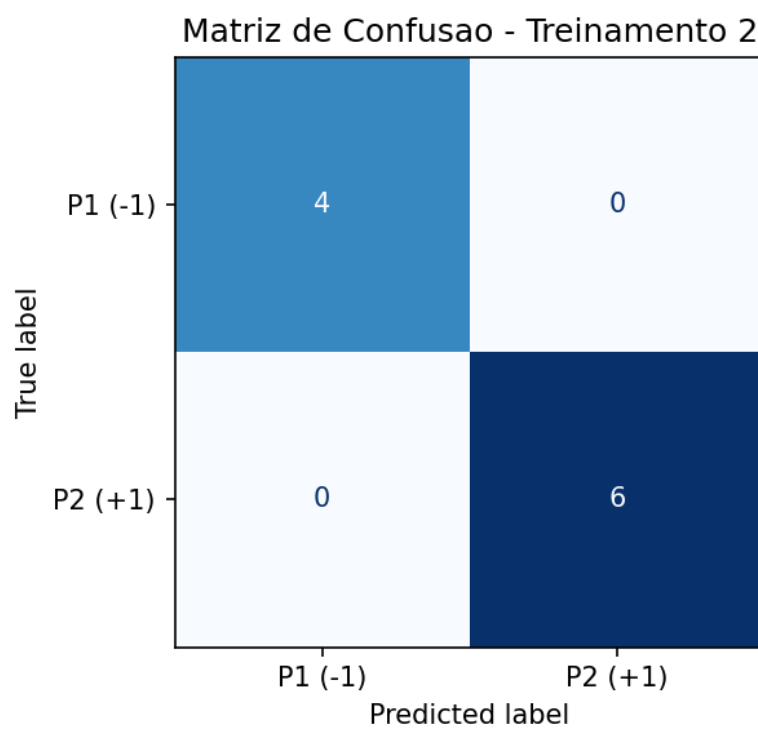


Figura 12 – Matriz de Confusão do 2º treinamento.

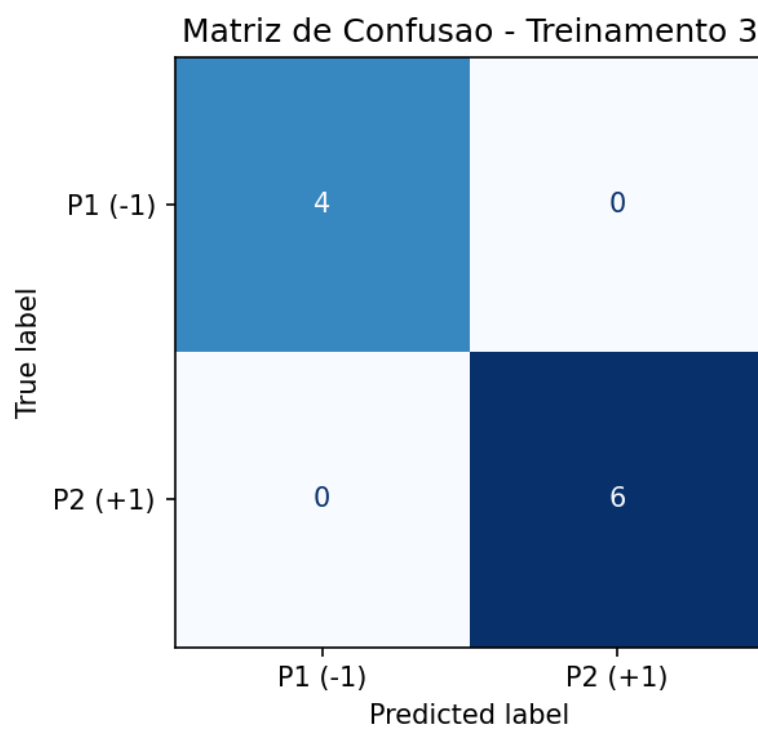


Figura 13 – Matriz de Confusão do 3º treinamento.

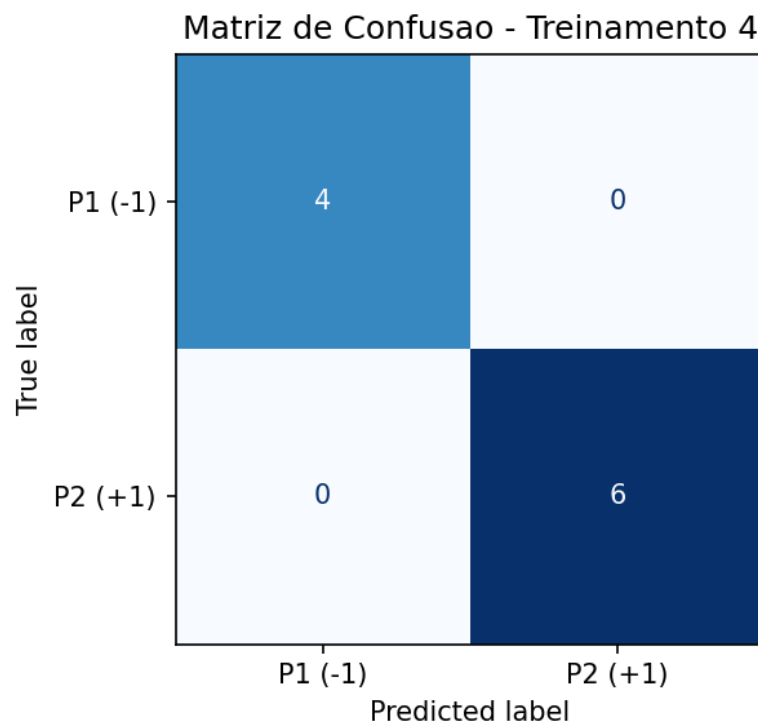


Figura 14 – Matriz de Confusão do 4º treinamento.

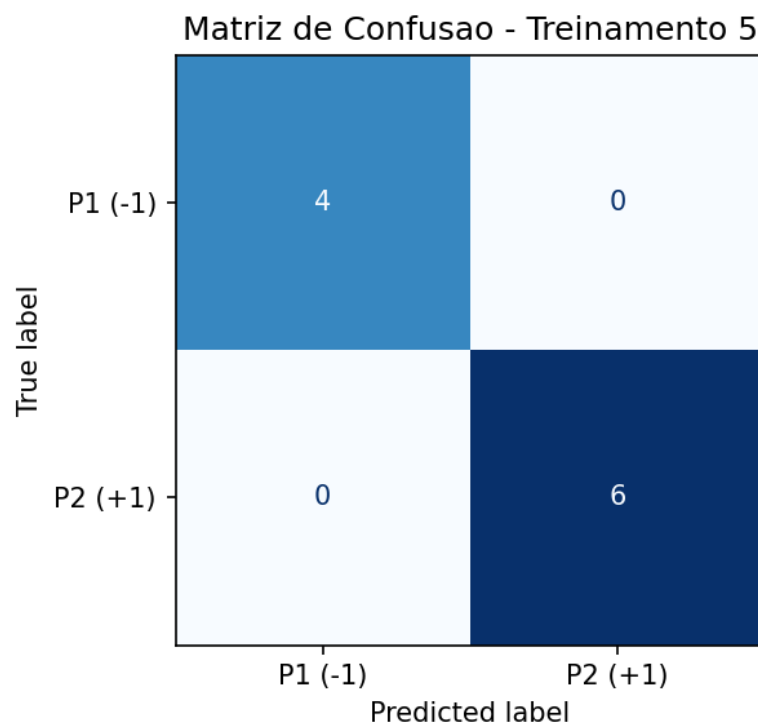


Figura 15 – Matriz de Confusão do 5º treinamento.

Observa-se que houve zero Falsos Positivos ($FP = 0$) e zero Falsos Negativos ($FN =$

0). Isso confirma que o hiperplano de separação encontrado pelo Perceptron foi capaz de isolar perfeitamente as amostras de classe P1 ($VN = 4$) e P2 ($VP = 6$) no espaço de características fornecido pela Tabela 1, alcançando acurácia máxima de 100% em todas as sessões.

6 Conclusão

O projeto demonstrou que o algoritmo Perceptron de camada única é uma ferramenta eficiente para resolver problemas de classificação binária. A implementação do código do zero em Python permitiu observar, na prática, como o aprendizado de máquina funciona. Adicionar um limite de segurança de 1000 épocas garantiu que o código não travasse, e a análise da métrica RMSE mostrou de forma clara como a rede reduz os erros gradativamente até aprender por completo, o que ocorreu entre 387 e 432 épocas.

As métricas geradas com a biblioteca *scikit-learn* confirmaram a qualidade e a capacidade de generalização do modelo. O classificador acertou 100% das amostras no conjunto de validação, alcançando pontuação máxima (1,0) em acurácia, sensibilidade, precisão e especificidade. Esses resultados provam que as características do petróleo analisadas são linearmente separáveis. Além disso, ficou claro que o Perceptron consegue encontrar vários hiperplanos de separação válidos, dependendo apenas dos valores aleatórios sorteados no início do treinamento.

Por fim, para consultar os detalhes do código, incluindo a automação das planilhas e a geração dos gráficos, todo o material do projeto está salvo e disponível publicamente no GitHub. O repositório pode ser acessado pelo link: <https://github.com/P3nido/Perceptron>.